

SC2079

MULTIDISCIPLINARY DESIGN PROJECT

Algorithms Briefing

Huang Shell Yíng
(assyhuang@ntu.edu.sg)

MDP TASK

Build a robotic system that can

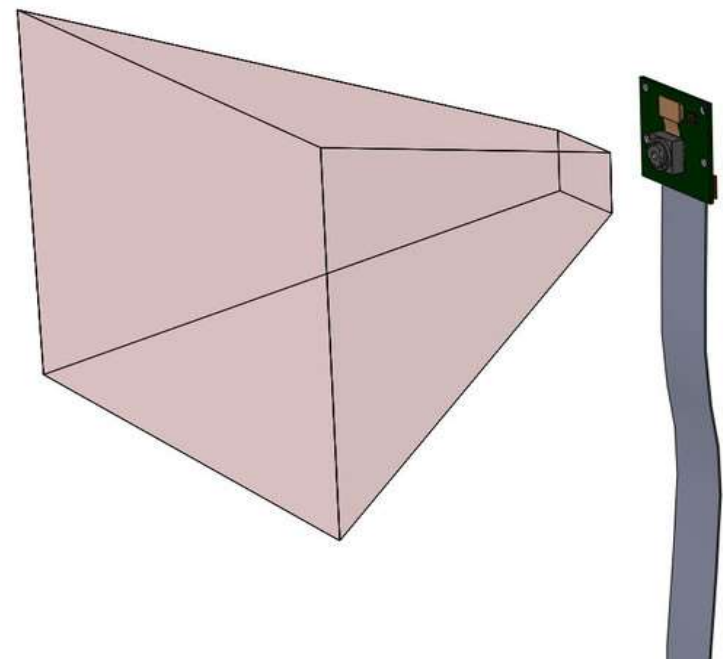
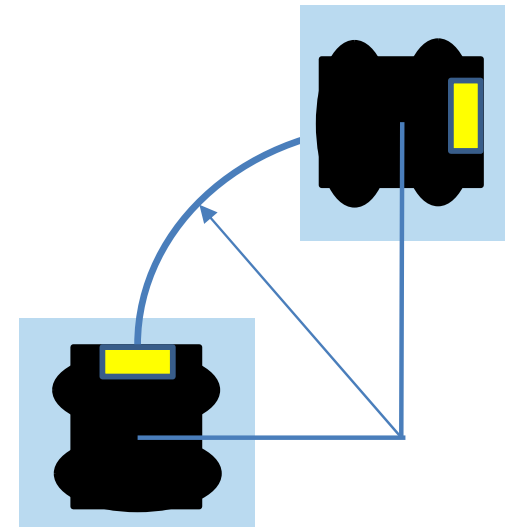
- Autonomously traverse a fixed area with known obstacles to recognize images pasted on the obstacles
- Transmit and receive control signals from mobile device
- Simulate the physical robot and algorithms in software

Robot's Environment

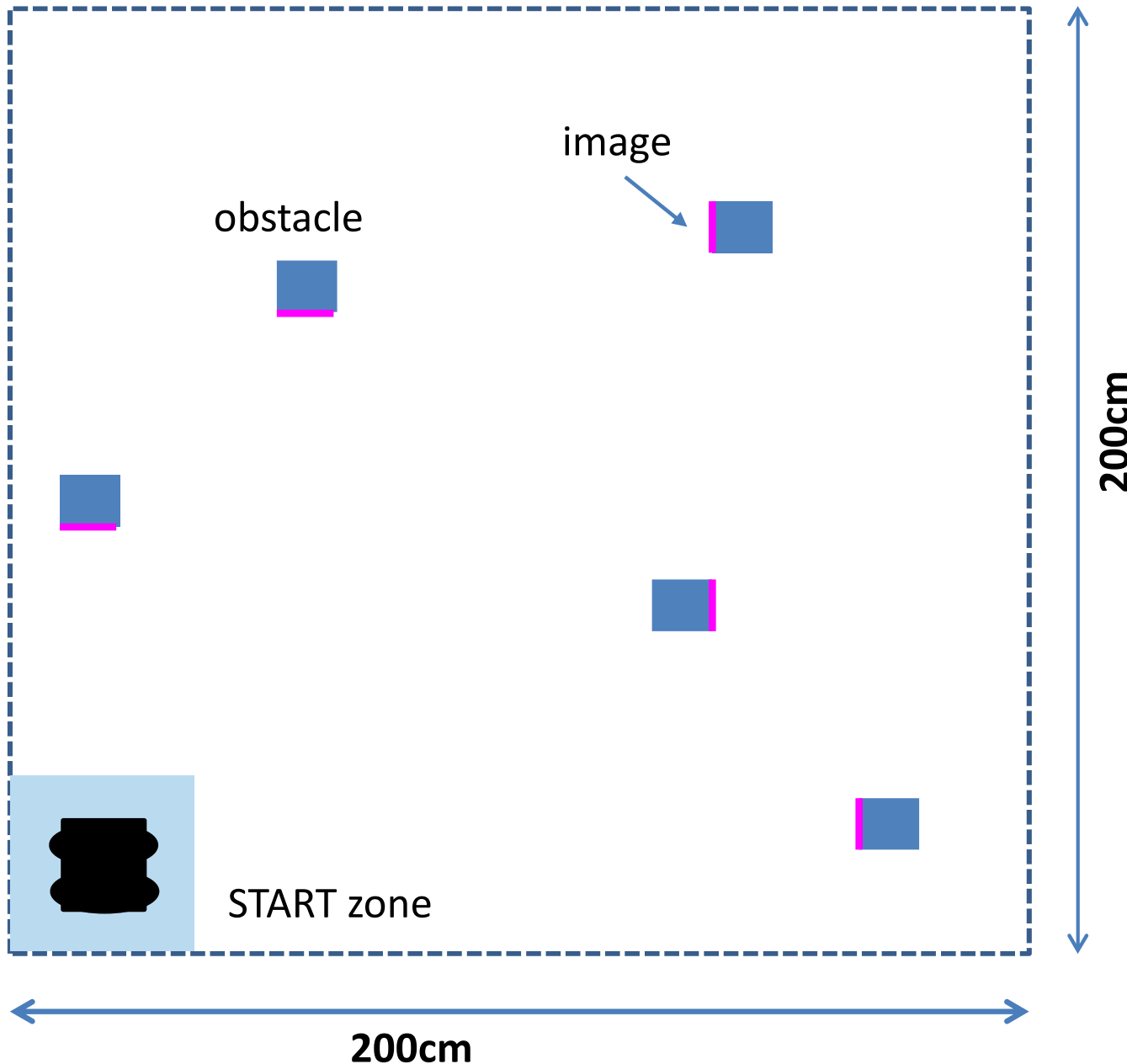
- A square area of 200cm x 200cm with virtual boundaries.
- There are five obstacles. Each is a block with 10cm x 10cm footprint scattered in the area.
- The obstacles are in the orientation that is parallel to the sides of the square area.
- An image is displayed on one of the four sides of an obstacle. Each of the other three sides has a 'bull eye' marker.
- The bottom left corner of the 200cm x 200cm area is the START zone of the robot (40cm x 40cm).
- Robot has a footprint of 20cm x 21cm with a camera fixed at its front center position.

Robot's Environment

- The 4-wheel robot can move forward, move backward, turn while moving.
- There is a turning radius of about 25cm but it is a larger radius if robot moves faster.
- The robot's camera has a conical field of vision. The best position is to have the camera 20cm away from the obstacle to recognize an image.



Sample Environment (top view)



No image will be in a location not accessible by the robot.

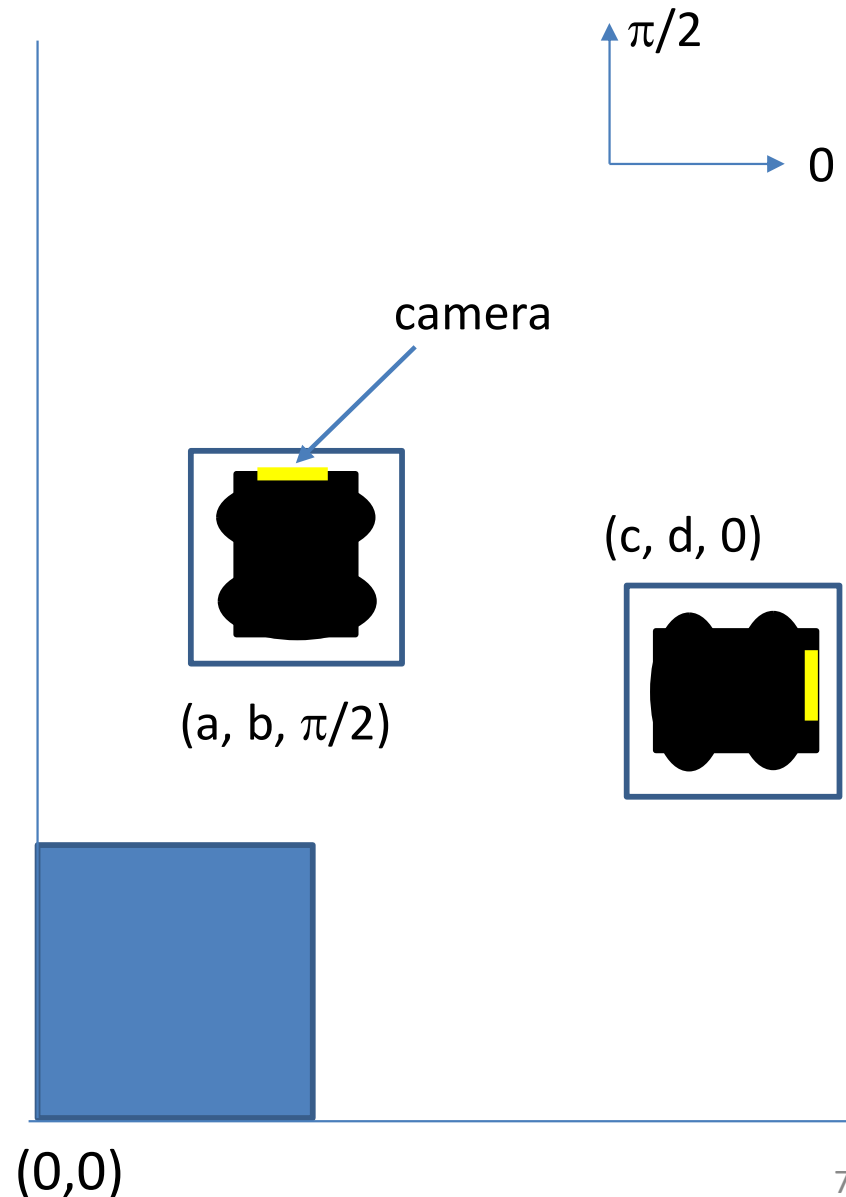
An image is any of the 15 images in the given set.

Problems/tasks to solve

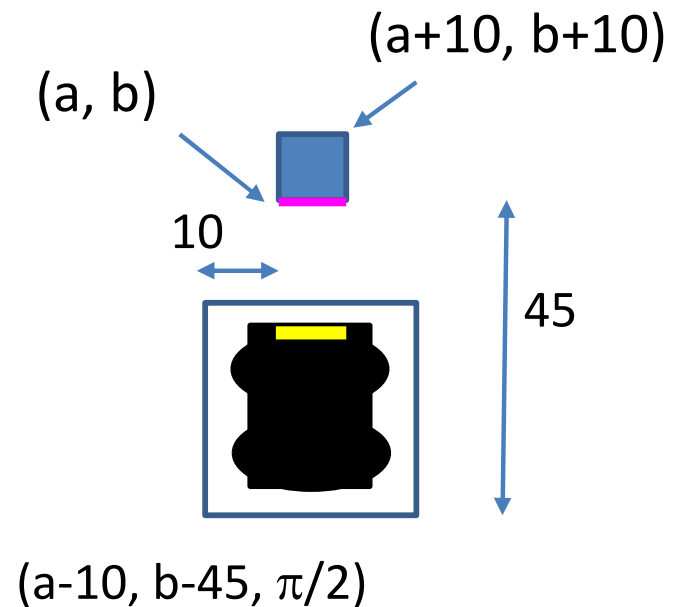
1. How to represent the navigational area, the robot, the obstacles and images
2. How to find a (shortest/fastest) Hamiltonian path from START point

1. Representations of the navigational area, the robot, the obstacles and image locations (option 1)

- 1) The navigational area: 200x200cm area.
- 2) Robot's start zone: a square with the bottom left coordinates at (0,0).
- 3) Recommended robot footprint to be used: 30cmx30cm.
- 4) Robot's location can be identified by the coordinates of the bottom left corner and its facing. (or by the robot's center)
- 5) Robot's facing is θ , where $-\pi < \theta \leq \pi$ with East facing to be 0.



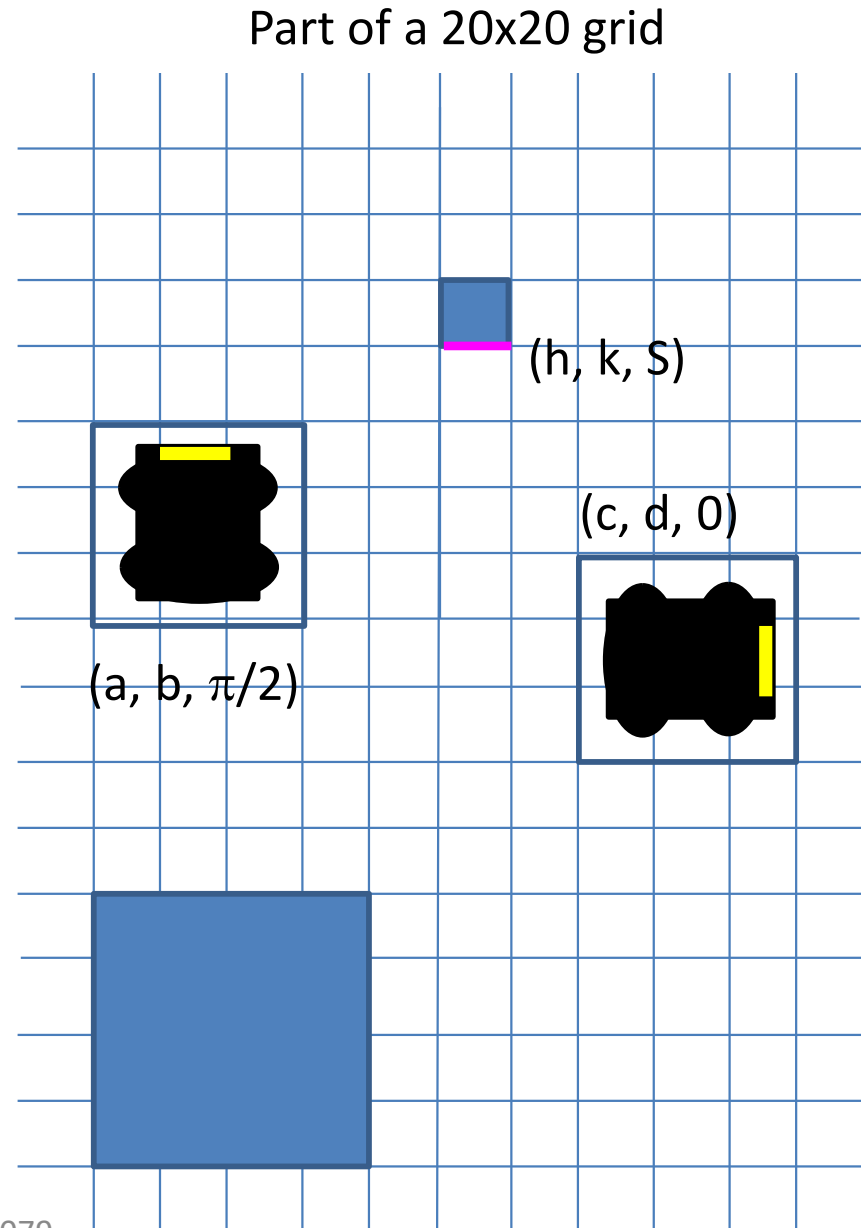
- 6) An obstacle's position is identified by its bottom left corner (h,k) . An image's position is identified by: (h, k, F) where $F = E, N, W, S$
- 7) Position the robot in front of an image for recognition
- If an image is located at (a, b, S) , we should target the robot at $(a-10, b-45, \pi/2)$
 - The center of the robot does not have to be aligned exactly with the center of the image/obstacle.



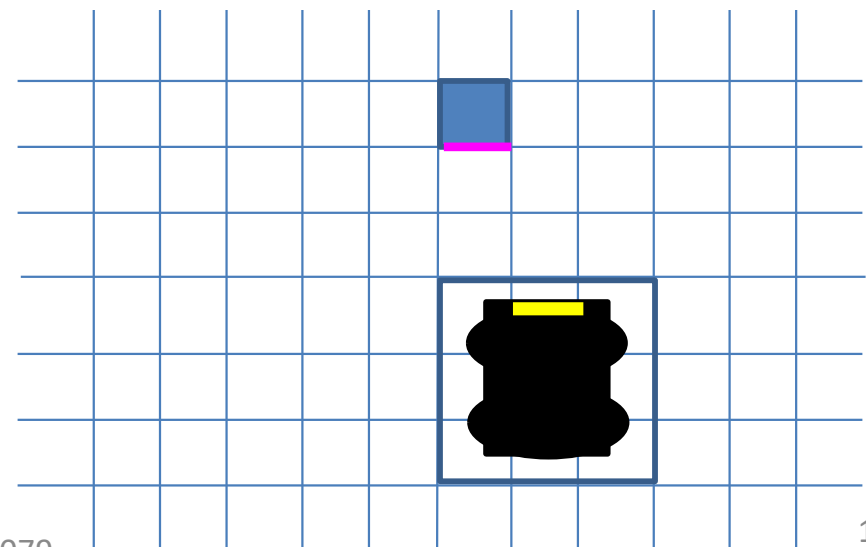
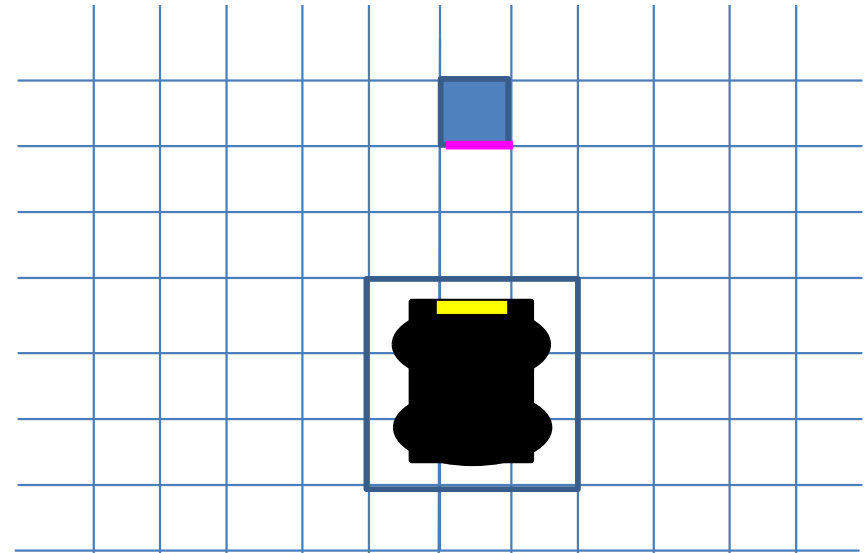
So the image represented by the pink line is at (a, b, S)

1. Representations of the navigational area, the robot, the obstacles and image locations (option 2)

- 1) By using a 20x20 grid so 10x10cm grid cells are used (or 40x40 grid of 5x5cm grid cells)
- 2) Robot's start zone: the bottom left corner is at cell (0,0)
- 3) Robot has a footprint of 30x30cm – 3x3 grid cells,
- 4) Robot's location can be identified by the bottom left grid cell and facing, e.g. $(a, b, \pi/2)$
- 5) An obstacle has a footprint of 10x10cm – 1x1 grid cell
- 6) E.g. an obstacle is located at cell (h,k) with the image on its south side : (h, k, S)



- 7) Position the robot in front of an image for recognition
- If the image is located at (a, b, S) , we should target the robot's bottom left corner at $(a-1, b-5, \pi/2)$
 - The center of the robot does not have to be aligned exactly with the center of an image/obstacle.



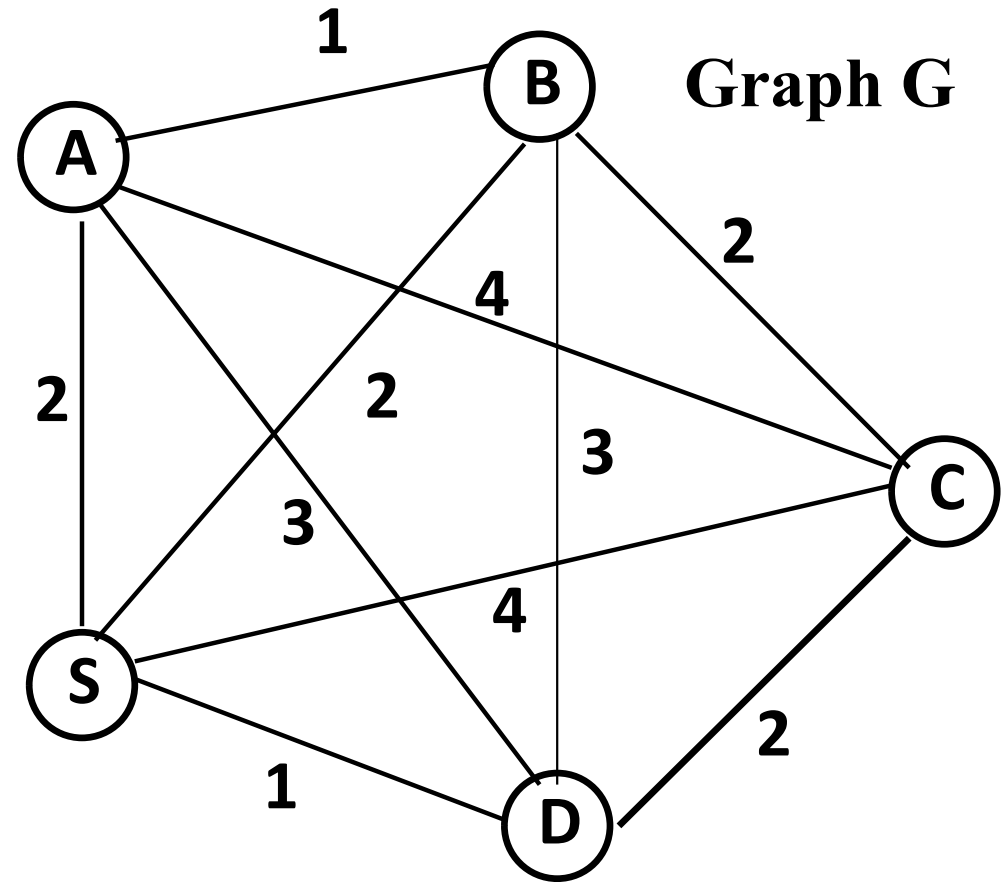
2. How to find the shortest/fastest Hamiltonian path from START point

Our Hamiltonian path problem:

Given a graph G , find a path, that starts from S and goes to all the vertices of the graph exactly once.

Three issues:

- 2.1 Planning a Hamiltonian Path
- 2.2 Path planning between 2 robot configurations
- 2.3 What if the image is not found



2.1 Planning a Hamiltonian Path

- This is to plan the whole path to travel to all images from the start point
 - 1) Trajectory between 2 robot configurations
 - 2) Basic path planning using greedy heuristic
 - 3) Improvement from basic path planning
 - 4) Planning the path using exhaustive search

1) Trajectory between 2 robot configurations

- Planning a Dubins path. We discuss this in Section 2.2

2) Basic path planning using greedy heuristic

nearestNeighbour()

{ $v = S$; initialize Visited[j] = false for all locations;

put v into Queue;

repeat 5 times {

 Compute trajectory from v to all w where

 Visited[w] is not true and select the nearest w

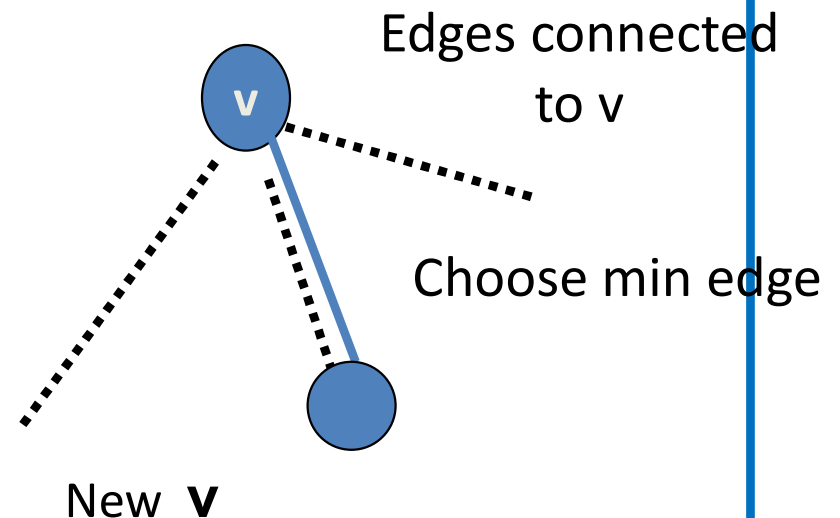
 put w into Queue;

 Visited[w] = true;

$v = w$; }

return Queue; }

See slides 41-42



3) Improvement from basic path planning (optional)

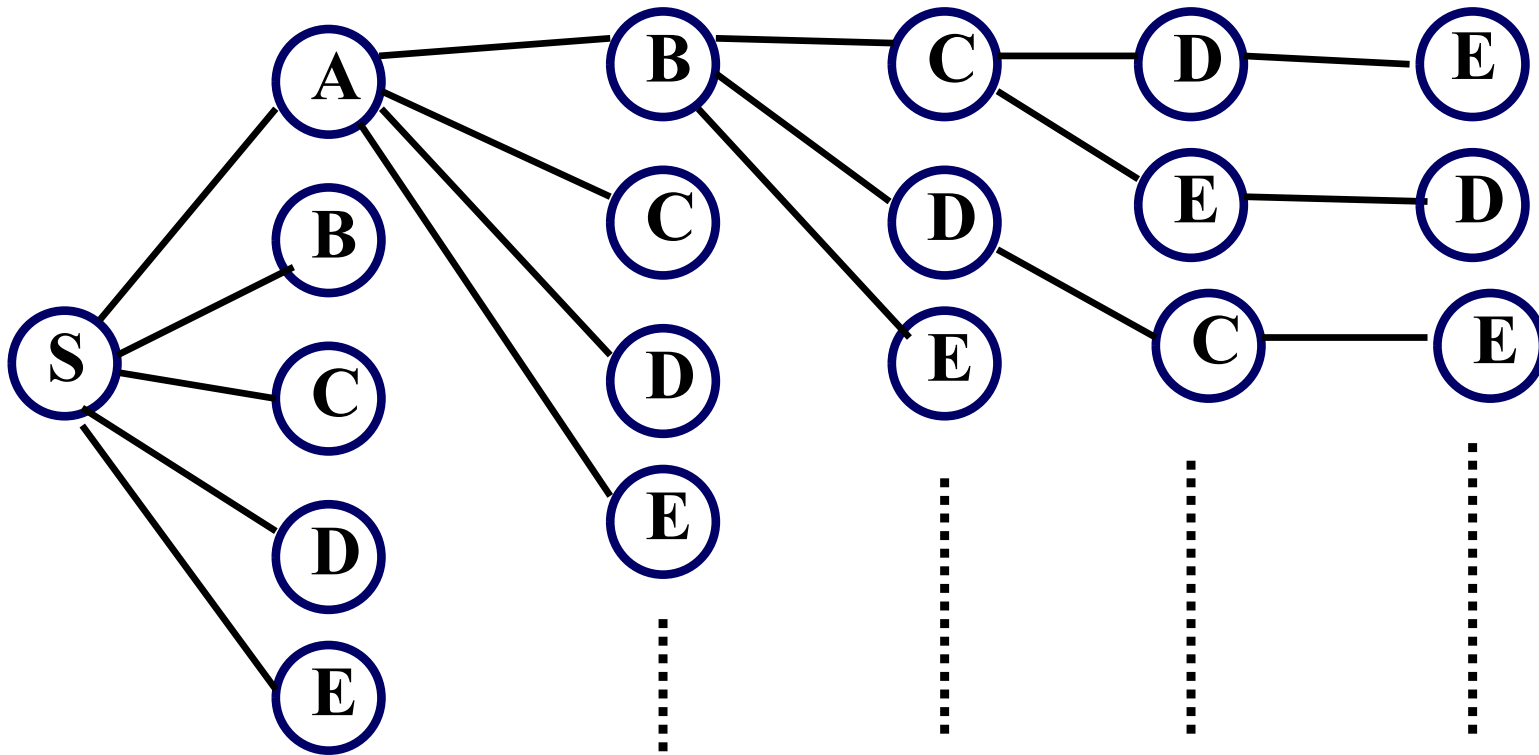
- After obtaining a path from the nearest-neighbour algorithm, try to swap between two vertices to see whether a better path may be obtained.

E.g. The nearest-neighbour algorithm returns a path SBCAD, try swapping BC, or CA or AD. That is, try SCBAD, SBACD, SBCDA

- Another improvement you may do is in the step “Compute trajectory from v to all w where $\text{Visited}[w]$ is not true and select the nearest w ”,
 - Order the vertices in non-decreasing order of the Euclidean distance for checking
 - If the Euclidean distance between v and w is already longer than the current shortest Dubin's path distance, there is no need to compute Dubin's path for this w

4) Planning using exhaustive search

Consider all possible paths to find the shortest/fastest path.



Given that we have only 5 obstacles to visit, we can afford the cost of the exhaustive search

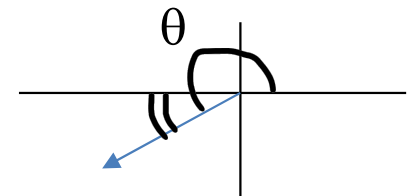
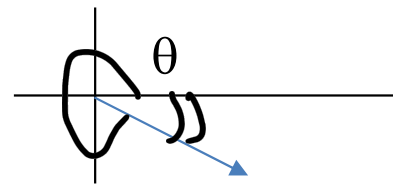
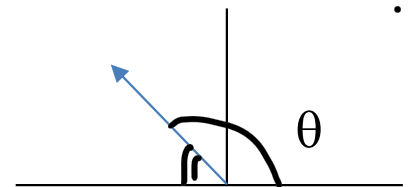
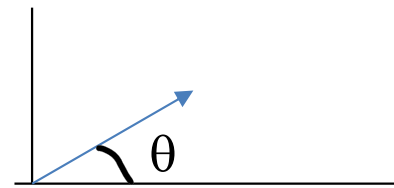
2.2 Path Planning between two robot configurations

- Car (robot) dynamics: when the robot is turning, we approximate the true system dynamic in small discrete time steps
- The robot's configuration can be described by (x, y, θ) where (x, y) is the robot's position and θ is the orientation of the robot.
- Δ is the distance the robot can move in a straight line in one time step
- Then in one time step, the robot's new configuration will be

$$x_{\text{new}} = x_{\text{prev}} + \Delta \cos(\theta)$$

$$y_{\text{new}} = y_{\text{prev}} + \Delta \sin(\theta)$$

$$\theta_{\text{new}} = \theta_{\text{prev}} + \frac{\Delta}{r}$$

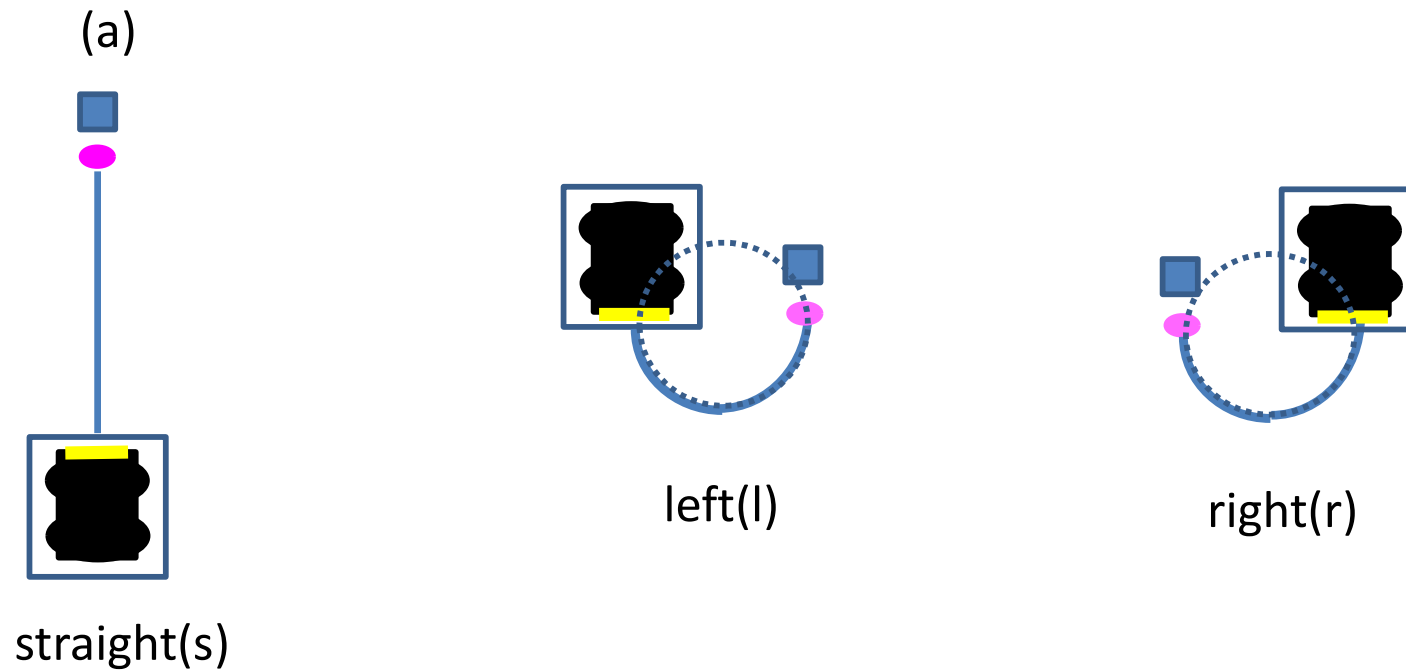


- We would like to plan a path to travel from the start point to an image or from an image to the next image in the shortest distance
 - 1) Consider a robot that can move forward, turn left or right with a turning radius. Path planning for a robot capable of reversing is more complicated
 - 2) Dubins (1957) proved that all shortest paths can be described by paths of up to three-segments consisting of arcs(part of a circle) or a straight line
 - 3) The arc is part of the circumference of a circle with the minimum turning radius
 - 4) The arc is denoted as C and the straight line S. Dubins proved that the three segments are of types **CSC** or **CCC** (rsr, rsl, lsr, lsl, rlr, lrl)

We will

- (i) look at what these paths look like;
- (ii) how to plan these paths;
- (iii) how to compute the lengths of these paths.

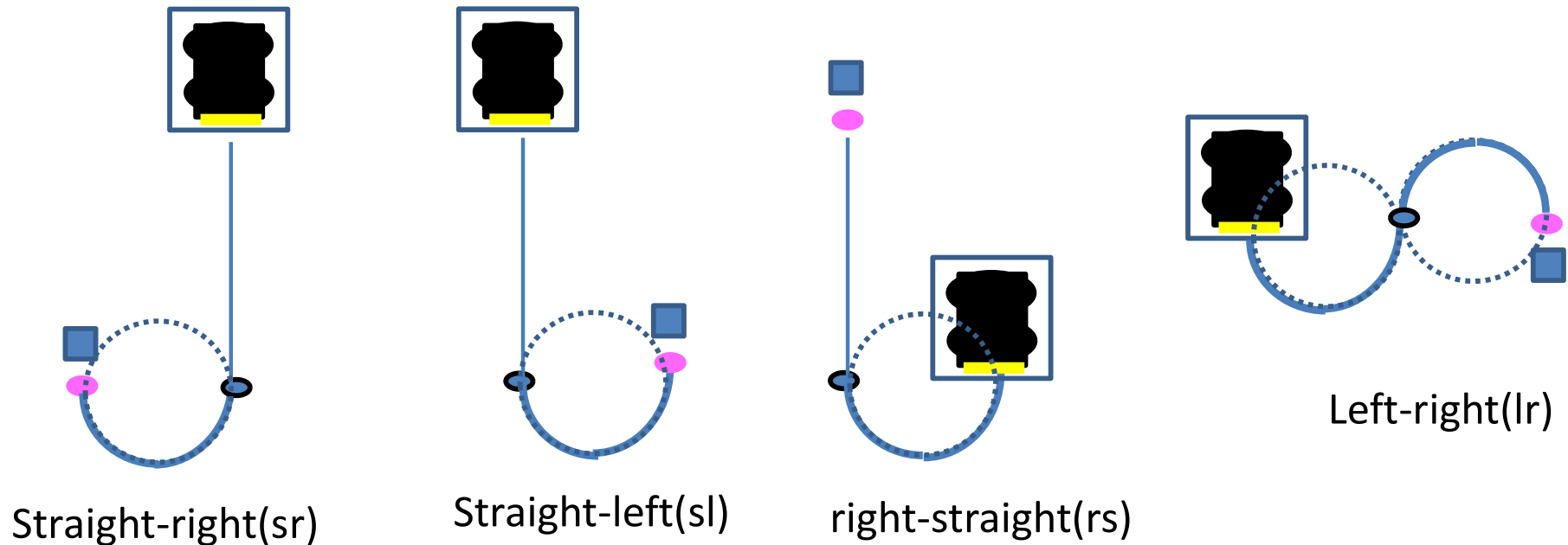
(i)(a) Path that has one segment (examples)



● Target position to go to

Target position is in front of image

(i)(b) Path that has two segments (examples)

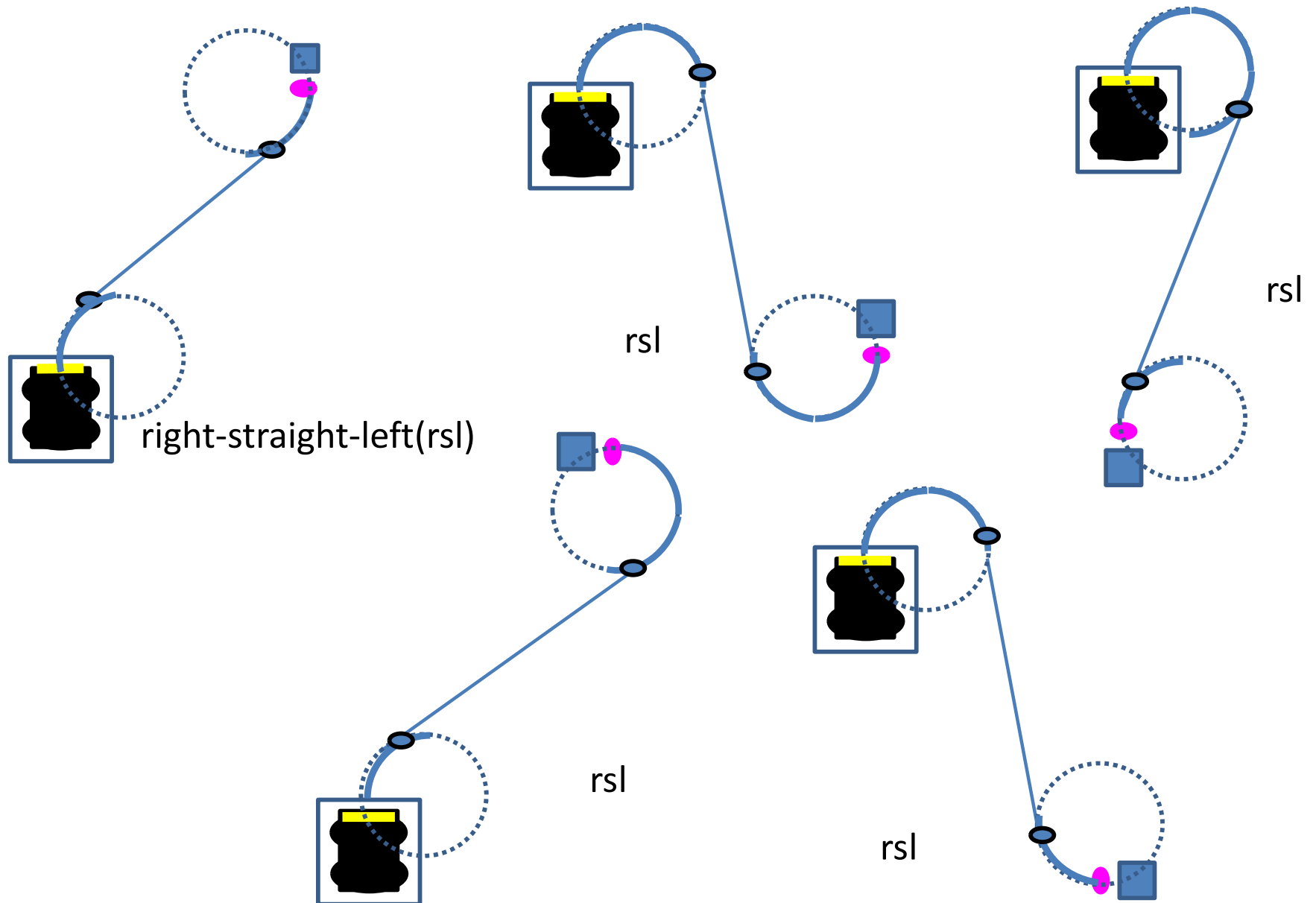


● Position to change segment

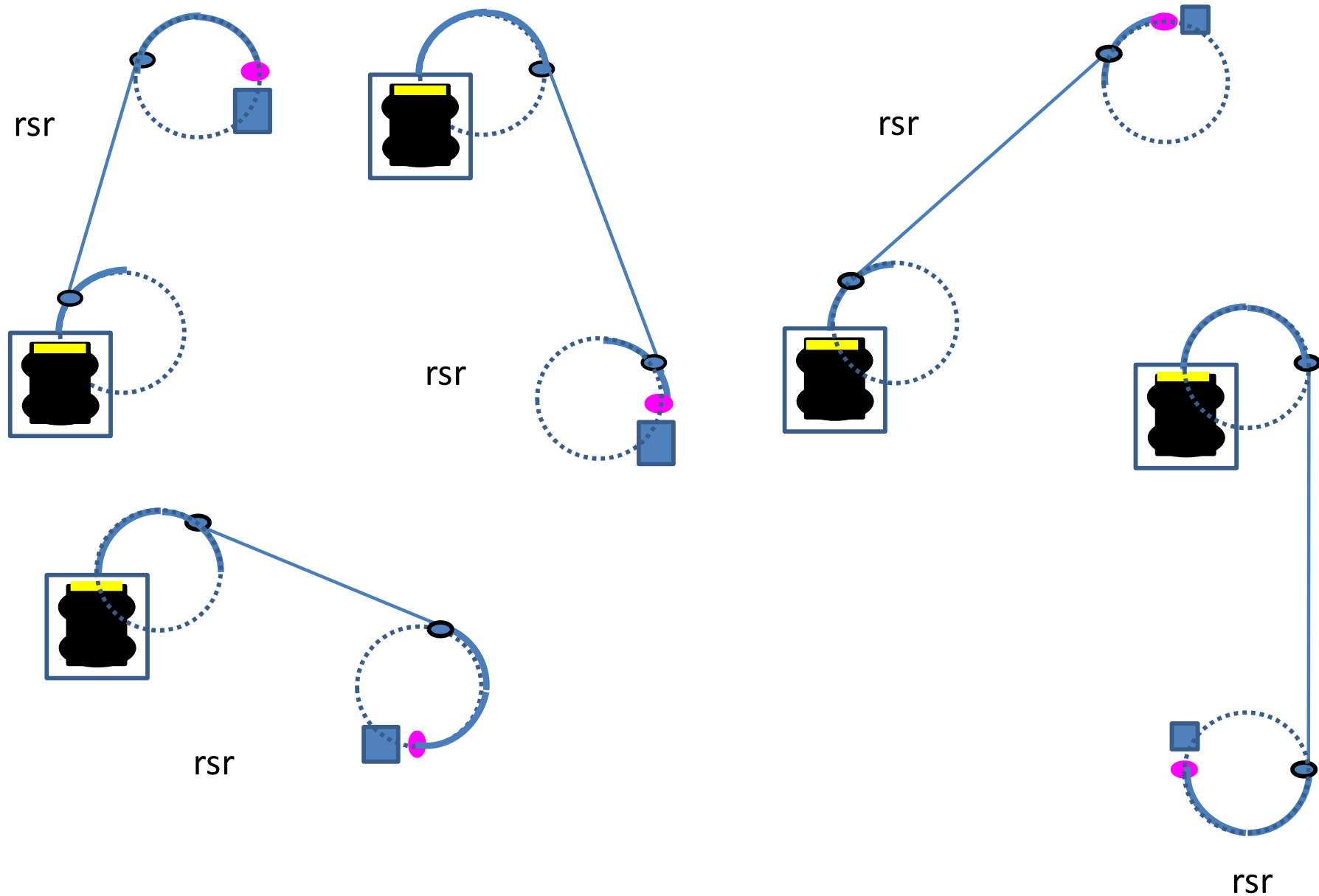
● Target position to go to

There are also ls
and rl paths

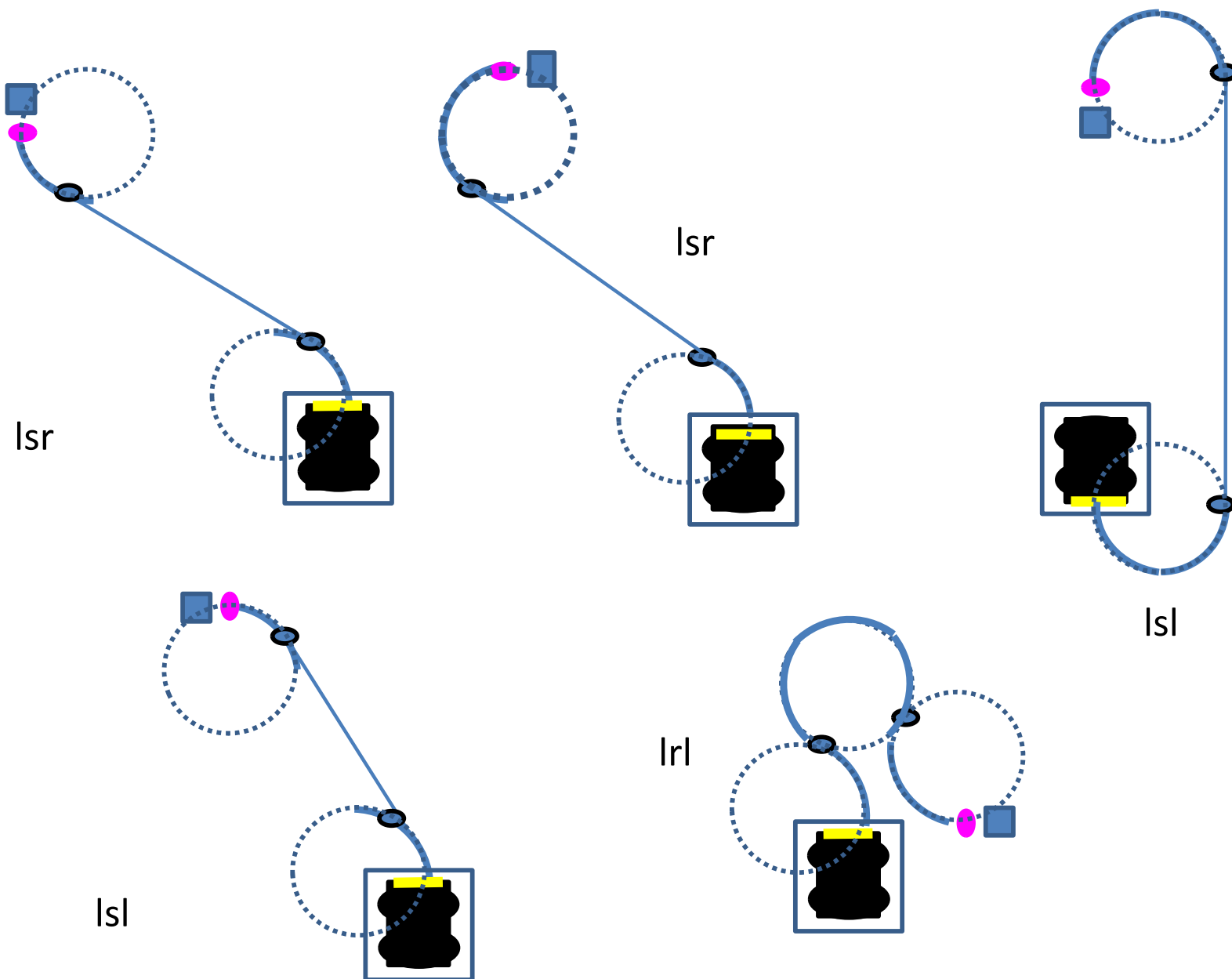
(i)(c) Path with three segments (examples)



(i)(c) Path with three segments (examples)

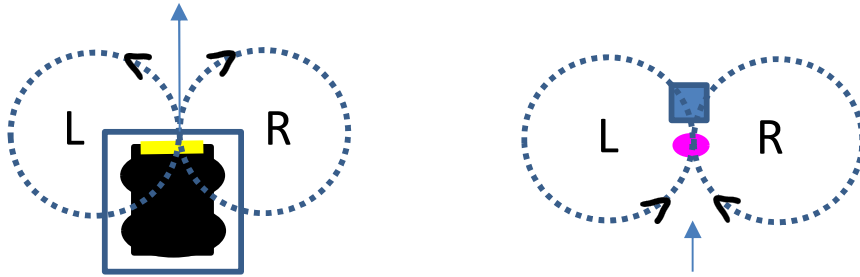


(i)(c) Path with three segments (examples)

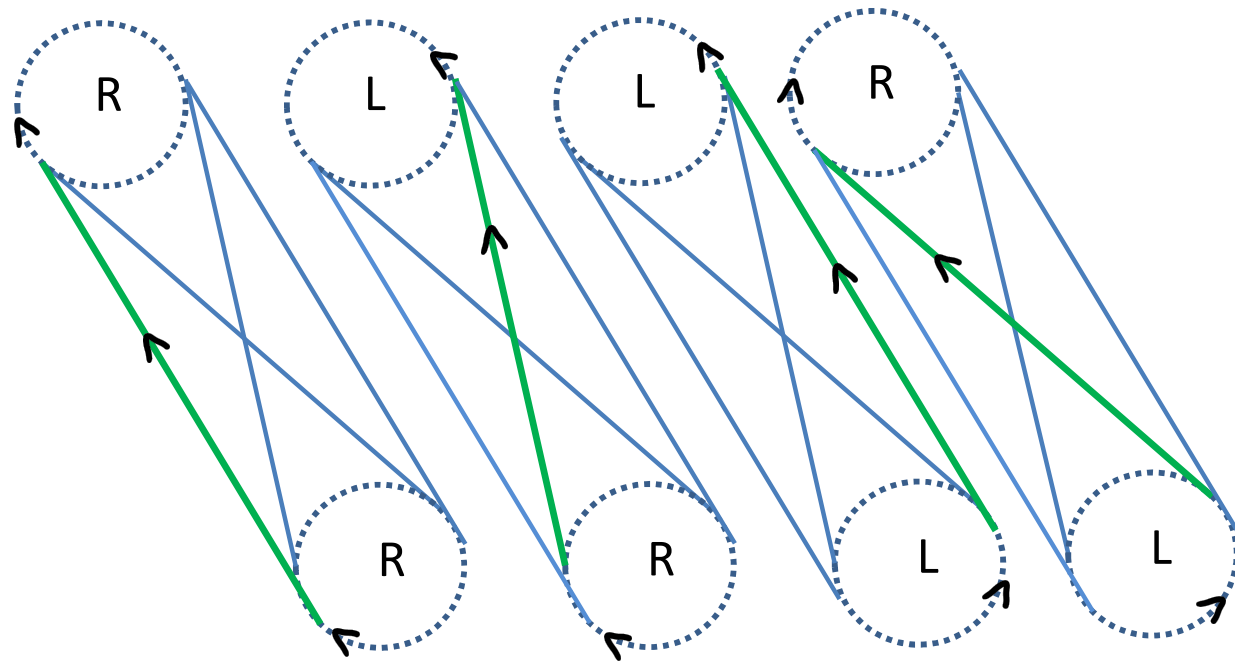


1. Rotate a vector (x, y) counterclockwise by $\pi/2$, we get $(-y, x)$
2. Rotate a vector (x, y) clockwise by $\pi/2$, we get $(y, -x)$
3. Reverse a vector (x, y) we get $(-x, -y)$
4. Rotate a vector (x, y) counterclockwise by angle δ , we get
 $(x * \cos(\delta) - y * \sin(\delta), x * \sin(\delta) + y * \cos(\delta))$

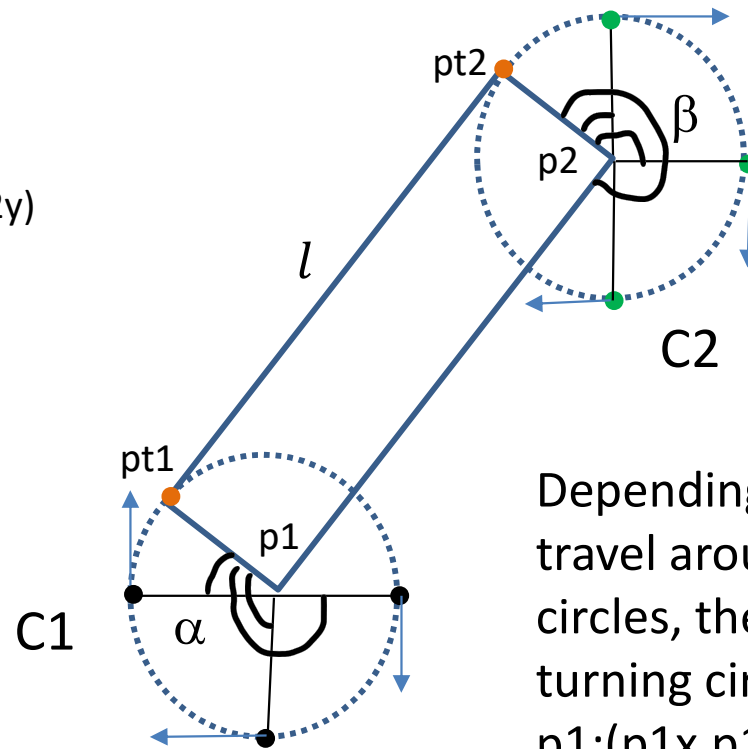
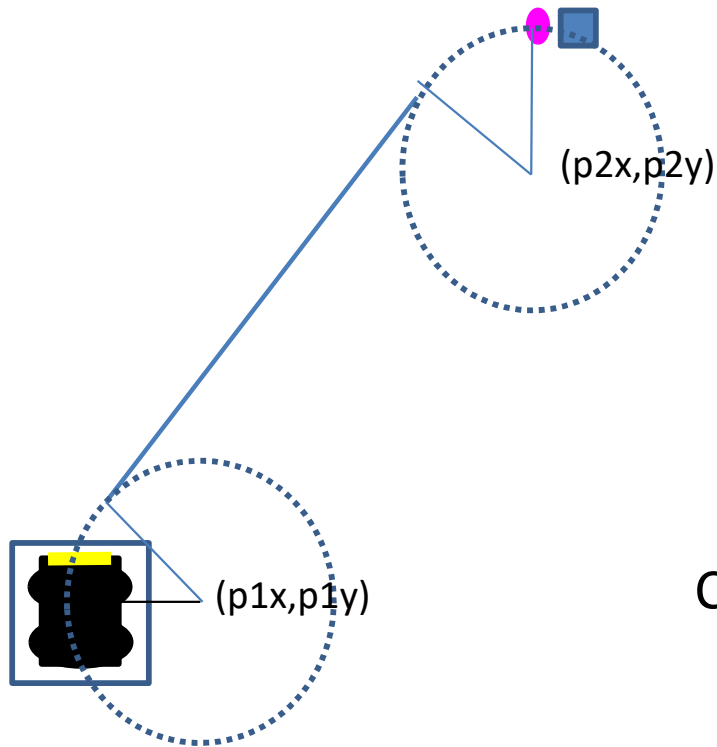
For CSC paths, we achieve the shortest path by turning along minimum-radius circles and longest straight line travel.



Between each pair of circles (RR), (RL), (LR), (LL), There are four tangent lines. There is only one valid tangent line for the robot's travel



(ii)(a) Path planning for rsr type

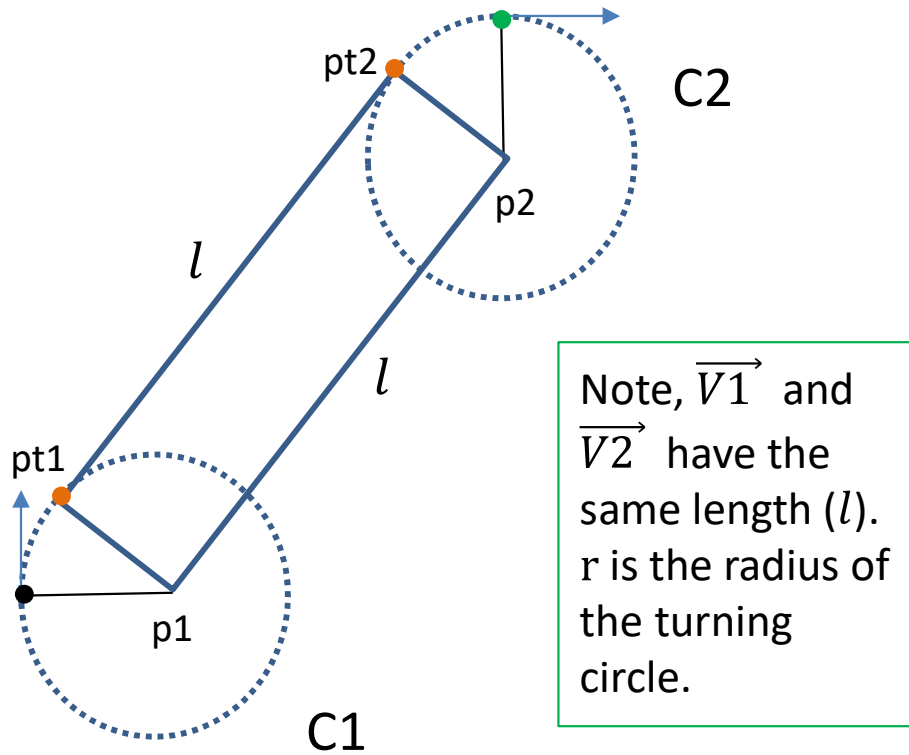


- Possible robot's position
- Possible target's position
- Transition point

Depending on the direction of travel around the turning circles, the centre of the two turning circles $C1$ and $C2$ at $p1:(p1x, p1y)$ and $p2:(p2x, p2y)$ can be calculated easily.

Robot's centre is at $(x1, y1)$.
 Target position is at $(x2, y2)$.
 Turning radius is r . The actual value you should use for r should be decided when you experiment with your robot.

To find the rsr path, we need to find the two turning angles α and β , and distance l . So we need to find the tangent points $pt1$ and $pt2$.



The s segment of the rsr path is the tangent line to both circles.

So l is the distance between the centres of the two turning circles.

$$l = \sqrt{(p2x - p1x)^2 + (p2y - p1y)^2}$$

The robot just needs to travel from its current position to pt1 then straight to pt2 and then to its target position.

The vector from p1 to p2, $\vec{V1}$
 $\vec{V1} = (V1x, V1y) = (p2x - p1x, p2y - p1y)$.

A vector $\vec{V2}$, counterclockwise rotation of $\vec{V1}$ by $\pi/2$, we get

$$\vec{V2} = (V2x, V2y) = (-V1y, V1x).$$

The point pt1, or the vector from (0,0) to pt1, is the vector from (0,0) to p1 plus the vector $r * \frac{\vec{V2}}{\|\vec{V2}\|}$.

$$\begin{aligned} \text{So } pt1x &= p1x + \frac{r}{l} * V2x \\ pt1y &= p1y + \frac{r}{l} * V2y \end{aligned}$$

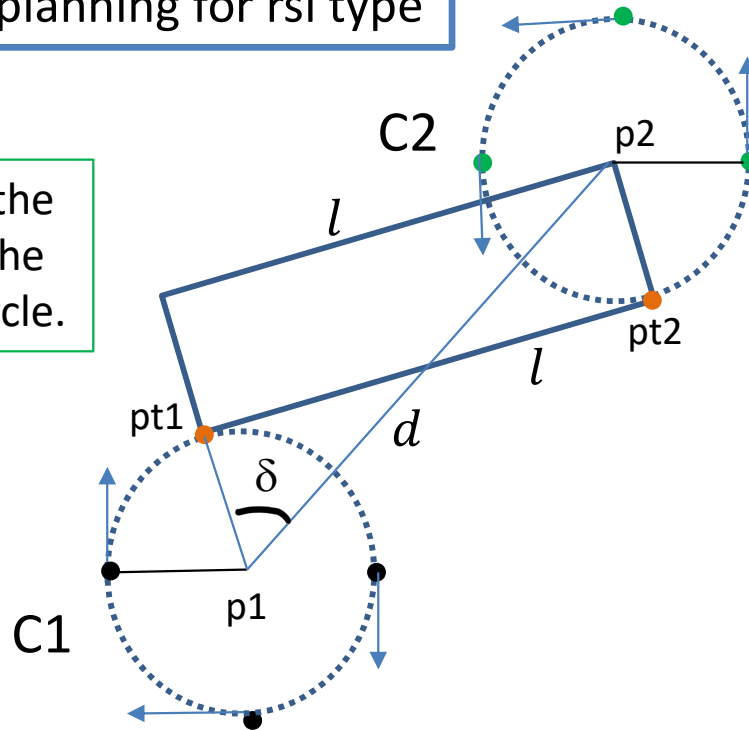
The point pt2, or the vector from (0,0) to pt2 is simply the vector from (0,0) to pt1 plus $\vec{V1}$.

So

$$\begin{aligned} pt2x &= pt1x + V1x \\ pt2y &= pt1y + V1y \end{aligned}$$

(ii)(b) Path planning for rsl type

Note, r is the radius of the turning circle.



The distance between the centres of the two turning circles.

$$d = \sqrt{(p2x - p1x)^2 + (p2y - p1y)^2}$$

The length of the straight line travel is

$$l = \sqrt{d^2 - (2r)^2}$$

The angle $\delta = \cos^{-1} \left(\frac{2r}{d} \right)$

The vector from $p1$ to $p2$, $\overrightarrow{V1}$

$$\overrightarrow{V1} = (V1x, V1y) = (p2x - p1x, p2y - p1y)$$

A vector $\overrightarrow{V2}$, counterclockwise rotation of $\overrightarrow{V1}$ by angle δ , we get

$$\begin{aligned} \overrightarrow{V2} &= (V2x, V2y) \\ &= (V1x * \cos(\delta) - V1y * \sin(\delta), \\ &\quad V1x * \sin(\delta) + V1y * \cos(\delta)) \end{aligned}$$

The point $pt1$, or the vector from $(0,0)$ to $pt1$, is the vector from $(0,0)$ to $p1$ plus the vector $r * \frac{\overrightarrow{V2}}{\|\overrightarrow{V2}\|}$

$$\begin{aligned} \text{So } pt1x &= p1x + \frac{r}{d} * V2x \\ pt1y &= p1y + \frac{r}{d} * V2y \end{aligned}$$

Reversing the direction of $\overrightarrow{V2}$

$$\overrightarrow{V3} = (V3x, V3y) = -\overrightarrow{V2} = (-V2x, -V2y)$$

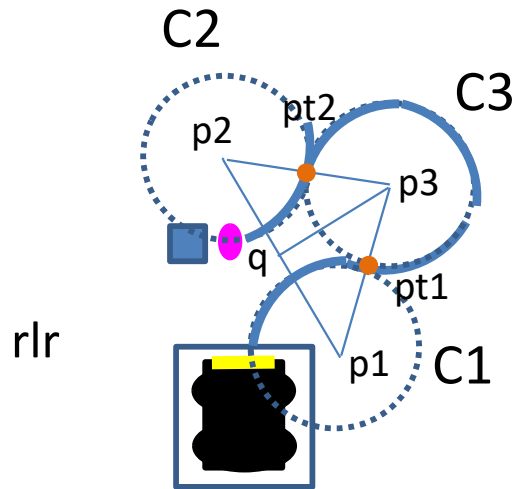
The point $pt2$, or the vector from $(0,0)$ to $pt2$ is simply the vector from $(0,0)$ to $p2$

plus $r * \frac{\overrightarrow{V3}}{\|\overrightarrow{V3}\|}$

$$\begin{aligned} \text{So } pt2x &= p2x + \frac{r}{d} * V3x \\ pt2y &= p2y + \frac{r}{d} * V3y \end{aligned}$$

(ii)(c) Path planning for lsr and lsl types will be similar.

(ii)(d) Path planning for rlr type



The path can be calculated by finding the centre $p3$ of the third circle $C3$.

The distance between the centres of the two turning circles $p1$ and $p2$.

$d =$

$$\sqrt{(p2x - p1x)^2 + (p2y - p1y)^2}$$

The middle point q between $p1$ and $p2$ is

$$q = ((p1x+p2x)/2, (p1y+p2y)/2)$$

The distance between $p1$ and $p3$ (and between $p2$ and $p3$) is $2r$.

The vector $\overrightarrow{V1}$ from $p1$ to $p2$,

$$\overrightarrow{V1} = (p2x - p1x, p2y - p1y)$$

A vector $\overrightarrow{V2}$, clockwise rotation of $\overrightarrow{V1}$ by $\pi/2$, is

$$\overrightarrow{V2} = (p2y - p1y, p1x - p2x)$$

The distance from $p3$ to q is

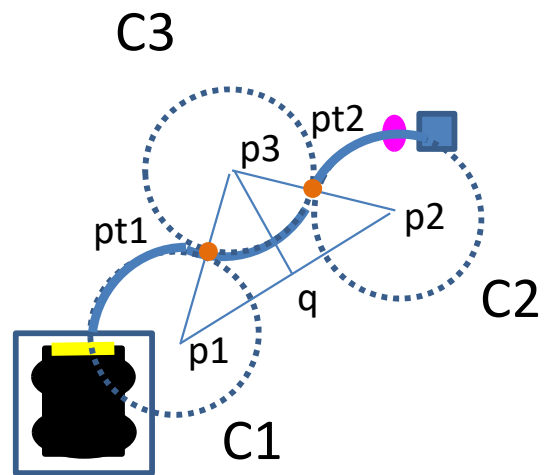
$$d1 = \sqrt{(2r)^2 - \left(\frac{d}{2}\right)^2}$$

The point $p3$, or the vector from $(0,0)$ to $p3$, is simply

the vector from $(0,0)$ to q plus $d1 * \frac{\overrightarrow{V2}}{\|\overrightarrow{V2}\|}$

After finding $p3$, the transition points $pt1$ and $pt2$ are the middle points between $p1$ and $p3$, between $p2$ and $p3$ respectively.

Another rlr



Everything is the same as the last slide except the calculation of $\overrightarrow{V2}$.

A vector $\overrightarrow{V2}$, counterclockwise rotation of $\overrightarrow{V1}$ by $\pi/2$, is

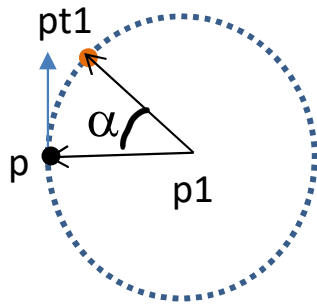
$$\overrightarrow{V2} = (p1y - p2y, p2x - p1x)$$

Note, it is possible to put C3 on the other side of C1, C2. But the path will be longer.

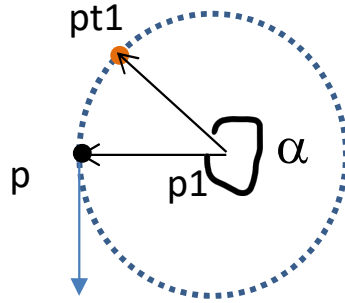
The CCC path is only useful when C1 and C2 are very close, i.e. the distance between them is less than $4r$. Otherwise a CSC path will be shorter.

Path planning for lrl type will be similar.

(iii) Computing Arc Length and path length



Turning right



Turning left

Point p is the source position and point pt1 is the target position that the robot will travel around the circle. The arc length depends on the direction of the travel.

Let $\vec{V1}$ be the vector from p1 to p and $\vec{V2}$ be the vector from p1 to pt1. Suppose p is at (x, y). Then

$$\vec{V1} = (V1x, V1y) = (x - p1x, y - p1y) \quad \text{and} \quad \vec{V2} = (V2x, V2y) = (pt1x - p1x, pt1y - p1y)$$

Calculate α by the code on the right.
The function `atan2()` is available in many languages.

$$\alpha = \text{atan2}(V2y, V2x) - \text{atan2}(V1y, V1x)$$

If $\alpha < 0$ and turning left

$$\alpha = \alpha + 2\pi$$

else if $\alpha > 0$ and turning right

$$\alpha = \alpha - 2\pi$$

Then the arc length = $|\alpha r|$

The path length is the sum of the two arc lengths and the length of the straight line travel for CSC paths, or the sum of the three arcs for CCC paths.

4) Obstacle avoidance

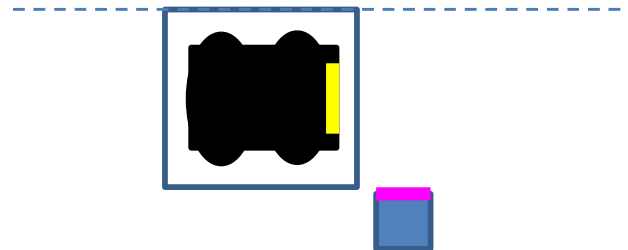
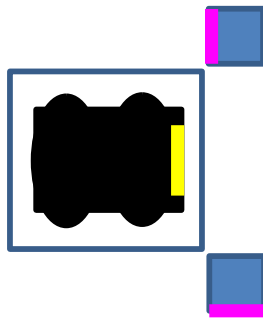
After the robot has recognized an image at an obstacle, this obstacle is blocking the robot – needs to reverse first



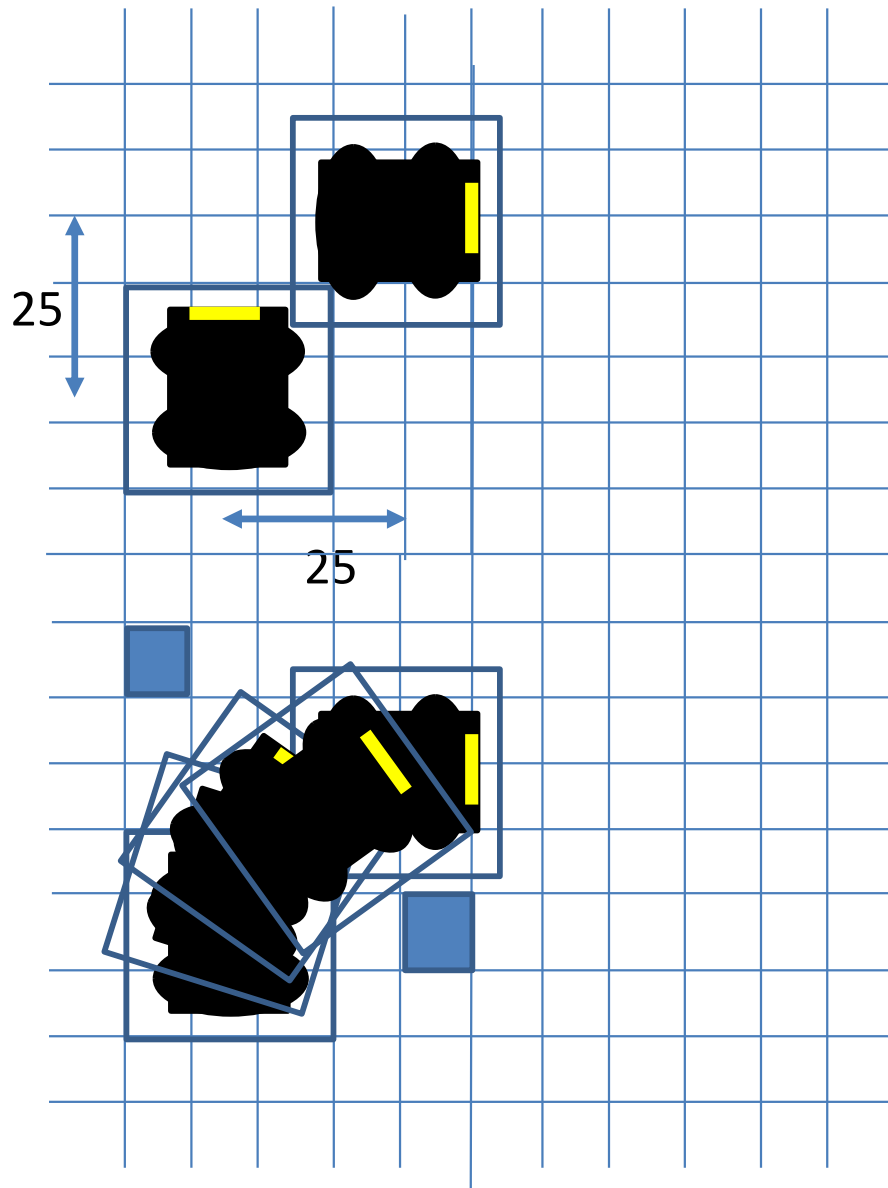
4) Obstacle avoidance

A path is not feasible if there is not enough space

- A straight line path needs a 30cm width between two obstacles and between an obstacle and the arena boundary
- A small detour can solve the problem but it will lengthen the path



4) Obstacle avoidance



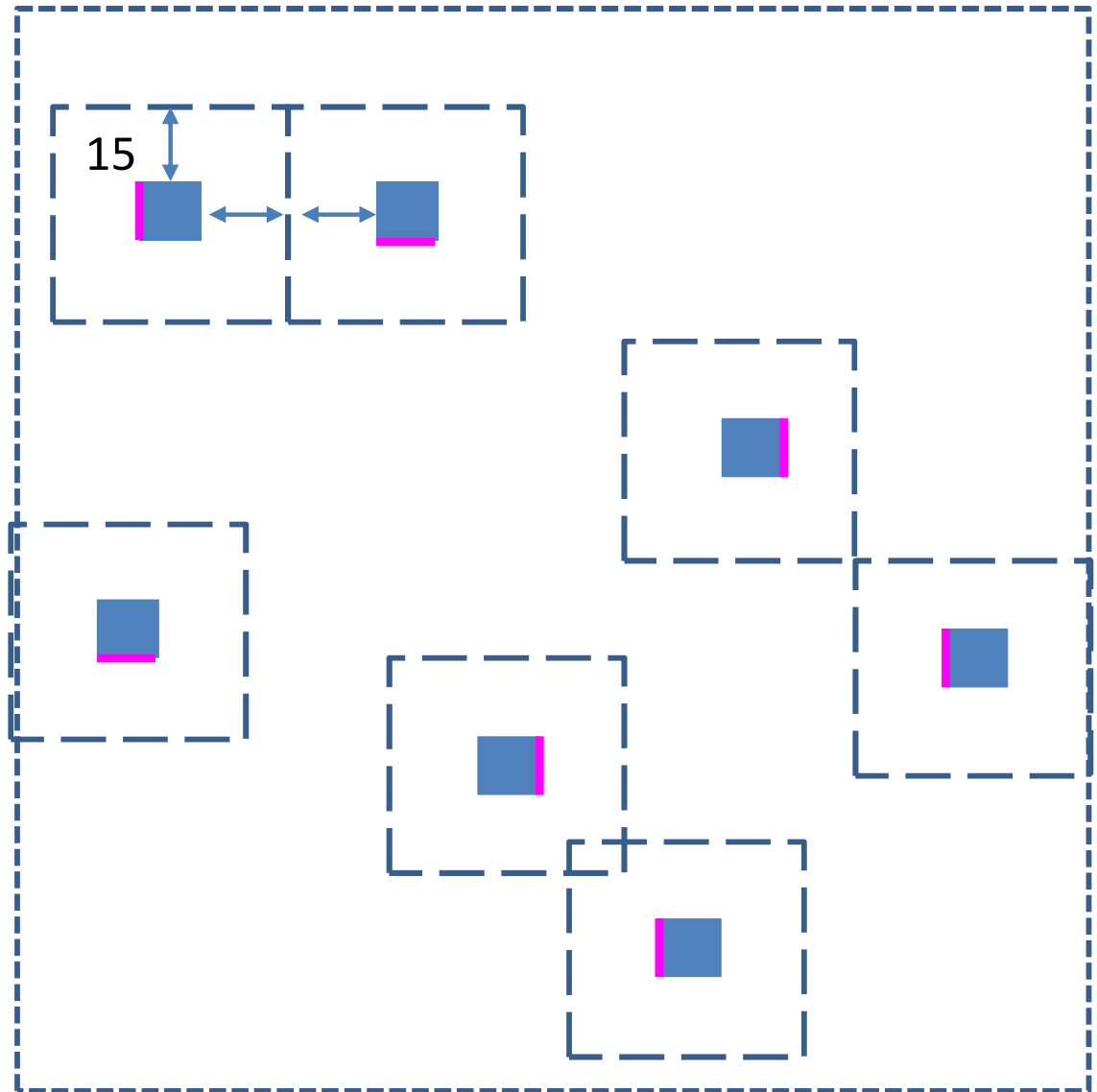
A path is not feasible if there is not enough space

- When the robot turns, it may go between two obstacles with their closest corners separated by 30cm in vertical and horizontal directions

A small detour can solve the problem but it will lengthen the path

4) Obstacle avoidance

- A simple way is to make virtual obstacles and consider the robot as a dot (the robot's center).
- The robot's footprint is 30cm x 30cm so the virtual obstacle should be 40cm x 40cm (not using the 20x20 grid cell representation of the arena)
- Then the robot's path should not get into any virtual obstacle's space

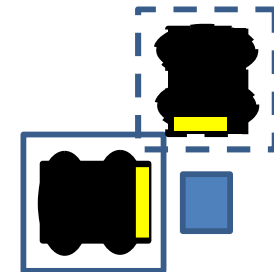


2.3 What if the image is not found

- If the robot's movement is not exactly done according to the planned path, the robot may not arrive at where it is supposed to be
 - 1) the robot may not find the image when it reaches an obstacle, instead a bull's eye is detected
 - 2) the robot may not see the obstacle where it expects to see one

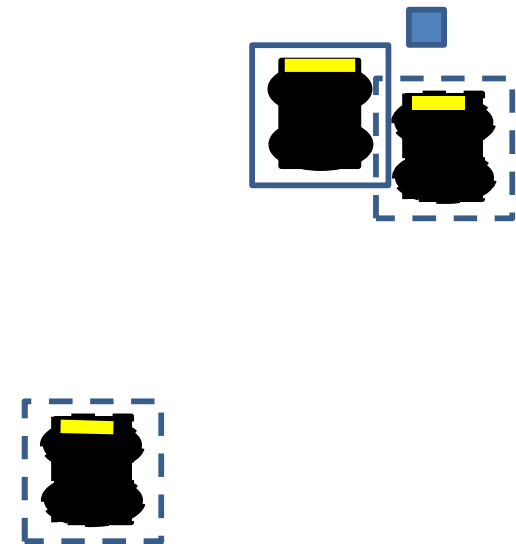
2.3 What if the image is not found

- 1) the robot may not find the image when it reaches an obstacle, instead a bull's eye is detected
 - a) Likely, the image is on a neighbouring side
 - b) Reverse and move around the obstacle
 - c) After the image is detected, continue with the next obstacle



2.3 What if the image is not found

- 2) the robot may not see the obstacle where it expects to see one
 - a) reverse and check since it is possible that the robot may overshoot
 - b) Else abandon the planned path, roam around (systematically) and go to any obstacle it may see



Simulate the physical robot and algorithms in software

- Display the 200cm x 200cm movement area with the start zone
- Display the obstacles and the image positions
- Display the robot's positions in real time (a square shape or a marker is ok) in a time stepped manner
- Report images recognized as it is done

Additional note: Alternative for finding the next nearest obstacle

- Using Euclidean distance to find the nearest neighbour is not accurate
 - Alternative is using the approach of Breadth First Search from current location (pink cell)
 - Assume the robot takes one time step to move one cell horizontally, vertically, or diagonally (again this is a simplifying assumption)
 - The blue cell is the cell to go in order to reach an obstacle
 - The number in a cell is the number of steps the robot takes to reach that cell from its current location (this is not the exact number of steps as the orientation/facing of the robot is not considered)
 - The nearest obstacle from the current location is the first cell the algorithm reaches
-
- In the example on the right, it is a simple path to reach the blue cell
 - So it takes 7 steps to reach that cell from its left
 - Since this is the first cell the algorithm reaches, this is the nearest neighbour

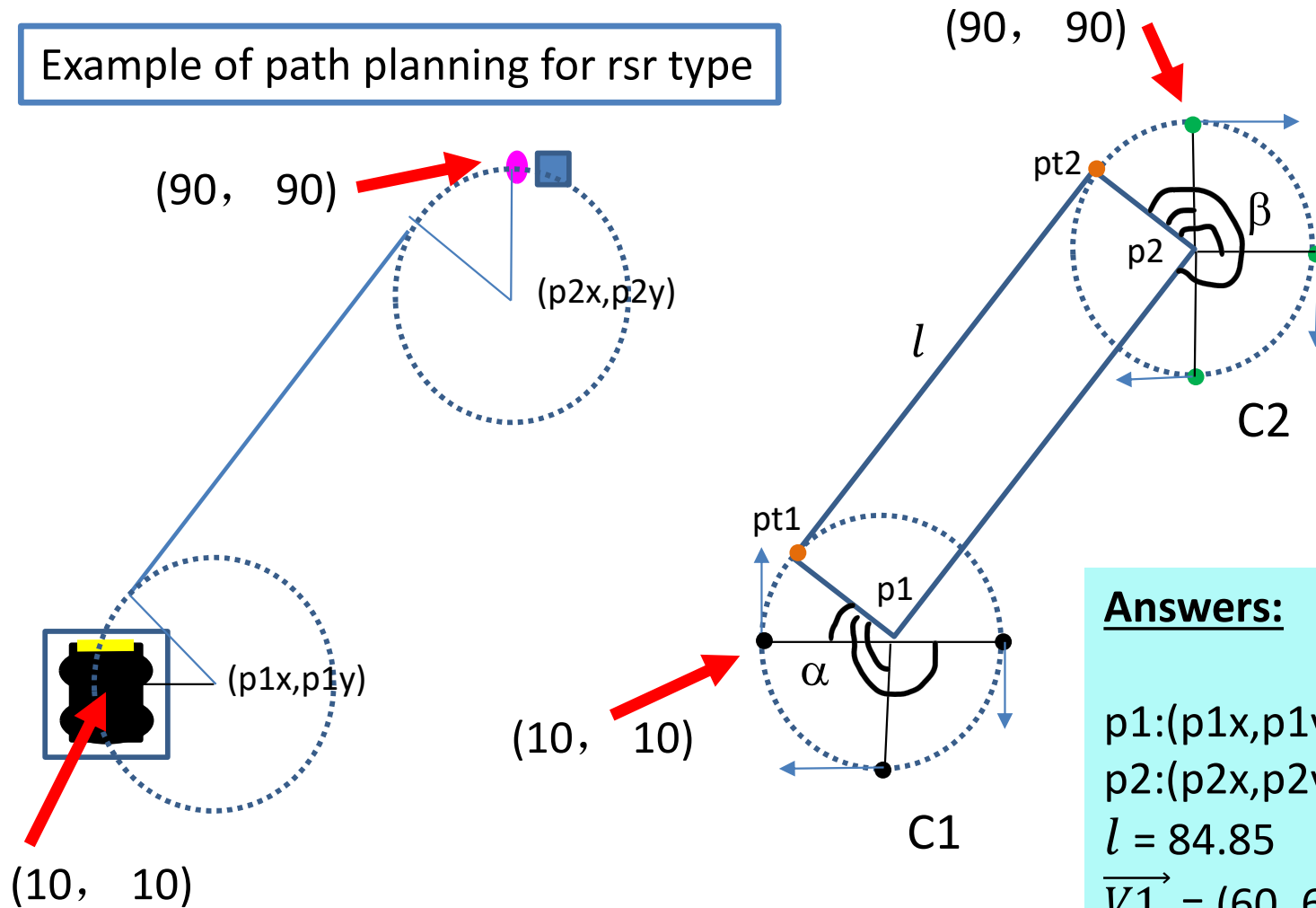
6	6	6	6	6	6	6	7		
5	5	5	5	5	5	6	7		
4	4	4	4	4	5	6	7		
3	3	3	3	4	5	6	7		
2	2	2	3	4	5	6	7		
1	1	2	3	4	5	6	7		
0	1	2	3	4	5	6	7		

Alternative for finding the next nearest obstacle

- In the example on the right, the grey cells are footprint of an obstacle visited.
- The yellow goal cell and the blue cell are two cells to go in order to reach 2 other (unvisited) obstacles respectively
- Using the approach of Breadth First Search, the yellow cell is found to be nearer than the blue cell even though by Euclidean distance, the blue cell is nearer.

1 3	1 2	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 2	
1 3	1 2	1 1	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	1 1	1 2	
1 3	1 2	1 1	0 0	9 9	9 9	9 9	9 9	9 9	9 9	1 0	1 1	1 2	
				8 8	8 8	8 8	8 8	8 8	9 9	1 0	1 1	1 2	
				7 7	7 7	7 7	7 7	8 8	9 9	1 0	1 1	1 2	
				6 6	6 6	6 6	7 7	8 8	9 9	1 0	1 1	1 2	
				5 5	5 5	6 6	7 7	8 8	9 9	1 0	1 1	1 2	
4 4	4 4	4 4	4 4	4 4	5 5	6 6	7 7	8 8	9 9	1 0	1 1	1 2	
3 3	3 3	3 3	3 3	4 4	5 5	6 6	7 7	8 8	9 9	1 0	1 1	1 2	
2 2	2 2	2 2	3 3	4 4	5 5	6 6	7 7	8 8	9 9	1 0	1 1	1 2	
1 1	1 1	2 2	3 3	4 4	5 5	6 6	7 7	8 8	9 9	1 0	1 1	1 2	
0 0	1 1	2 2	3 3	4 4	5 5	6 6	7 7	8 8	9 9	1 0	1 1	1 2	

Example of path planning for rsr type



- Possible robot's position
- Possible target's position
- Transition point

Answers:

$$p1:(p1x,p1y) = (30, 10)$$

$$p2:(p2x,p2y) = (90, 70).$$

$$l = 84.85$$

$$\overrightarrow{V1} = (60, 60)$$

$$\overrightarrow{V2} = (-60, 60)$$

$$pt1x = 15.85$$

$$pt1y = 24.14$$

$$pt2x = 75.86$$

$$pt2y = 84.14$$

Robot's centre is at (10, 10).
Target position is at (90, 90).
Turning radius is 20.